

1. O que é o Express.js e quais suas principais vantagens?

O **Express.js** é um framework para Node.js que simplifica a criação de aplicações web e APIs. Ele fornece recursos para roteamento, middlewares e integração com bancos de dados.

Principais vantagens:

- Código mais organizado e limpo;
- Criação rápida de APIs REST;
- Sistema de rotas simples;
- Suporte a middlewares;
- Grande comunidade e documentação;
- Fácil integração com MySQL, MongoDB e outros bancos.

2. Quais são os quatro principais tipos de rotas HTTP?

Os principais métodos HTTP são:

- **GET** → Consultar dados.
- **POST** → Inserir novos dados.
- **PUT** → Atualizar dados existentes.
- **DELETE** → Remover dados.

Exemplo:

```
app.get("/usuarios", ...);
```

```
app.post("/usuarios", ...);
```

```
app.put("/usuarios/:id", ...);
```

```
app.delete("/usuarios/:id", ...);
```

3. Explique o conceito de middleware no Express.js.

Middleware é uma função executada entre a requisição do cliente e a resposta do servidor.

Ele pode:

- validar dados;
- verificar autenticação;
- registrar logs;
- tratar erros;
- modificar a requisição ou resposta.

Exemplo:

```
app.use((req, res, next) => {  
  
  console.log(req.method, req.url);  
  
  next();  
  
});
```

4. Como criar uma conexão básica entre Node.js e MySQL?

Primeiro instala-se o pacote:

```
npm install mysql2
```

Depois cria-se a conexão:

```
const mysql = require("mysql2/promise");  
  
const conexao = await mysql.createConnection({  
  
  host: "localhost",  
  
  user: "root",  
  
  password: "",  
  
  database: "empresa"  
  
});
```

5. O que é um Prepared Statement e como ele protege contra SQL Injection?

Prepared Statements utilizam parâmetros em vez de concatenar valores diretamente na consulta SQL.

Exemplo:

```
const sql = "SELECT * FROM usuarios WHERE nome = ?";
```

```
const [resultado] = await conexao.execute(sql, [nome]);
```

Isso impede que comandos SQL enviados pelo usuário sejam executados como parte da consulta, protegendo contra **SQL Injection**.

6. Qual é a estrutura básica de um servidor Express.js?

```
const express = require("express");
```

```
const app = express();
```

```
app.get("/", (req, res) => {  
  res.send("Servidor funcionando!");  
});
```

```
app.listen(3000, () => {  
  console.log("Servidor iniciado.");  
});
```

Essa estrutura cria um servidor HTTP ouvindo na porta 3000.

7. Explique a diferença entre rotas globais e rotas específicas.

Rotas globais utilizam middlewares aplicados a toda a aplicação.

```
app.use(express.json());
```

Todas as rotas utilizarão esse middleware.

Já as **rotas específicas** aplicam middlewares apenas em determinadas rotas.

```
app.get("/admin", verificarLogin, (req, res) => {  
    res.send("Área administrativa");  
});
```

8. Liste três boas práticas ao manipular dados em bancos de dados.

- Utilizar Prepared Statements.
- Validar os dados recebidos do usuário.
- Tratar erros utilizando `try/catch`.

Também é recomendado:

- utilizar Pool de Conexões;
- armazenar credenciais no `.env`;
- evitar consultas SQL concatenadas.

9. Por que é importante usar variáveis de ambiente em aplicações Node.js?

As variáveis de ambiente permitem armazenar informações sensíveis fora do código.

Exemplo:

```
DB_HOST=localhost
```

```
DB_USER=root
```

```
DB_PASSWORD=123456
```

Benefícios:

- maior segurança;
- facilidade de configuração;
- ambientes separados (desenvolvimento, homologação e produção);
- evita exposição de senhas no GitHub.

10. Como tratar erros em consultas SQL dentro do Node.js?

Utilizando `try/catch`.

```
try {
```

```
    const [usuarios] = await conexao.query("SELECT * FROM usuarios");
```

```
    console.log(usuarios);
```

```
} catch (erro) {
```

```
    console.error("Erro:", erro.message);
```

```
}
```

Também é possível retornar respostas apropriadas para a API.

```
res.status(500).json({
```

```
    erro: erro.message
```

```
});
```